EECS 247
May 3rd, 2026

Memory-Efficient Fine-Tuning with Sparse Adapters (MEFT)

George Joseph Elassal

---

1. Motivation and Literature

Fine-tuning large language models (LLMs) is computationally expensive, particularly in terms of GPU memory usage. Traditional fine-tuning requires updating all model parameters, which can involve hundreds of millions or even billions of weights. This makes it impractical on limited hardware.

To address this, parameter-efficient fine-tuning methods such as adapters have been introduced. Adapters add small trainable modules to a frozen base model, significantly reducing the number of trainable parameters. However, as adapter size (rank) increases to improve model capacity, GPU memory usage still grows and eventually becomes a bottleneck.

The MEFT (Memory-Efficient Fine-Tuning) method, proposed by Hao et al. (ACL 2024), addresses this limitation by exploiting sparsity in adapter activations. It observes that for a given input, only a small subset of adapter neurons are active, and therefore only those need to be computed.

---

2. Algorithmic Methods

The key idea behind MEFT is dynamic sparsity via Top-K selection.

Instead of computing the entire adapter, MEFT:

1. Stores the full adapter on the CPU
2. Computes an activation score for each neuron
3. Selects the Top-K most active neurons
4. Transfers only those neurons to the GPU
5. Performs computation using this subset

This reduces GPU memory usage significantly, since only a small fraction of the adapter is active at each step.

In the original paper, this selection is optimized using a Mixture-of-Experts (MoE) routing mechanism. In this project, a simplified version is implemented using direct Top-K selection.

---

3. Project Design

Model Setup

- Base model: GPT-2 Medium
- Base model parameters: frozen
- Adapter added in parallel to each feed-forward layer

Two Configurations

1. **Baseline Adapter**
    - Entire adapter stored and computed on GPU
    - All neurons used every step
2. **MEFT Adapter**
    - Adapter stored on CPU
    - Only Top-K neurons used per step
    - K = 64

Parameters Tested

- Adapter rank: 256 → 8192
- Metrics measured:
    - Peak GPU VRAM usage
    - Training speed (steps per second)

---

4. Program

The implementation modifies GPT-2 by replacing its feed-forward network (FFN) layers with a custom module: Code can be seen using git hub link: https://github.com/gelassal/MEFT-LLM-Sparse-Adapters

```python
output = frozen_ffn(h) + adapter(h)
```

For the MEFT adapter:

- Weight matrices are stored on CPU
- Activation scores are computed:

```python
scores = h_cpu @ WA^T
```

- Top-K neurons are selected:

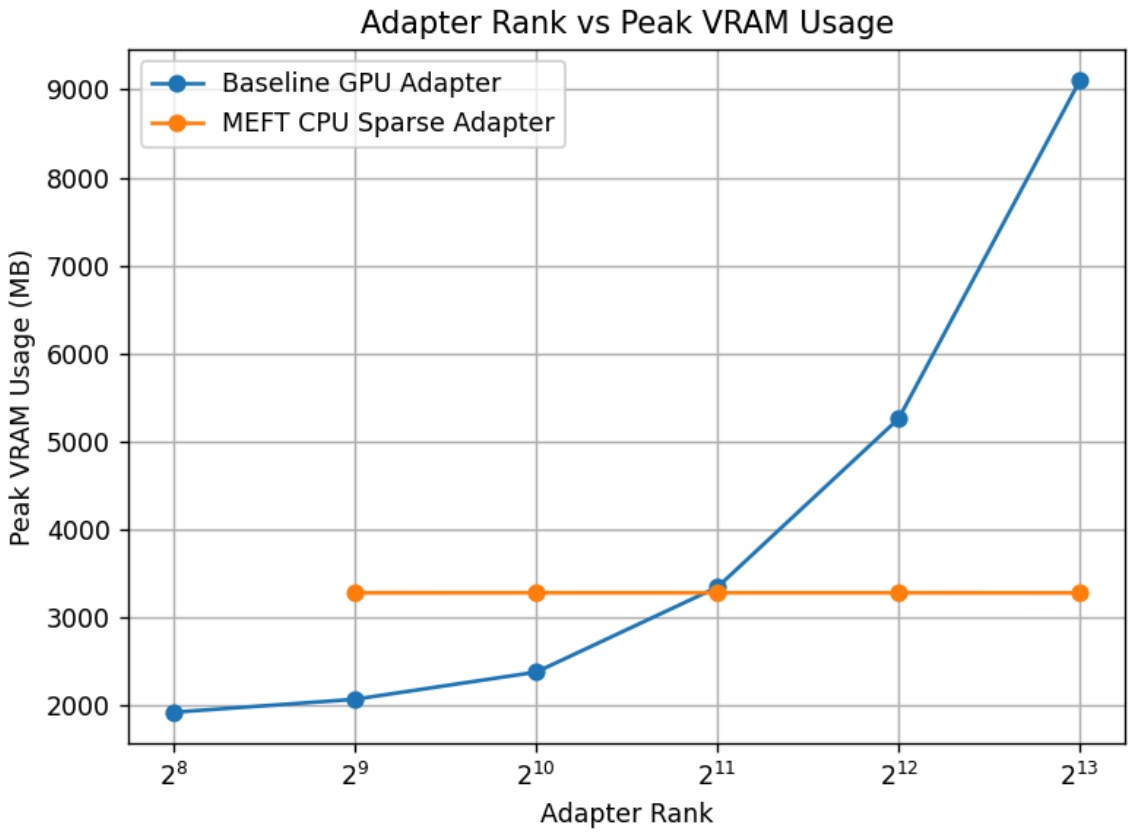
```python
topk_idx = torch.topk(scores, K)
```

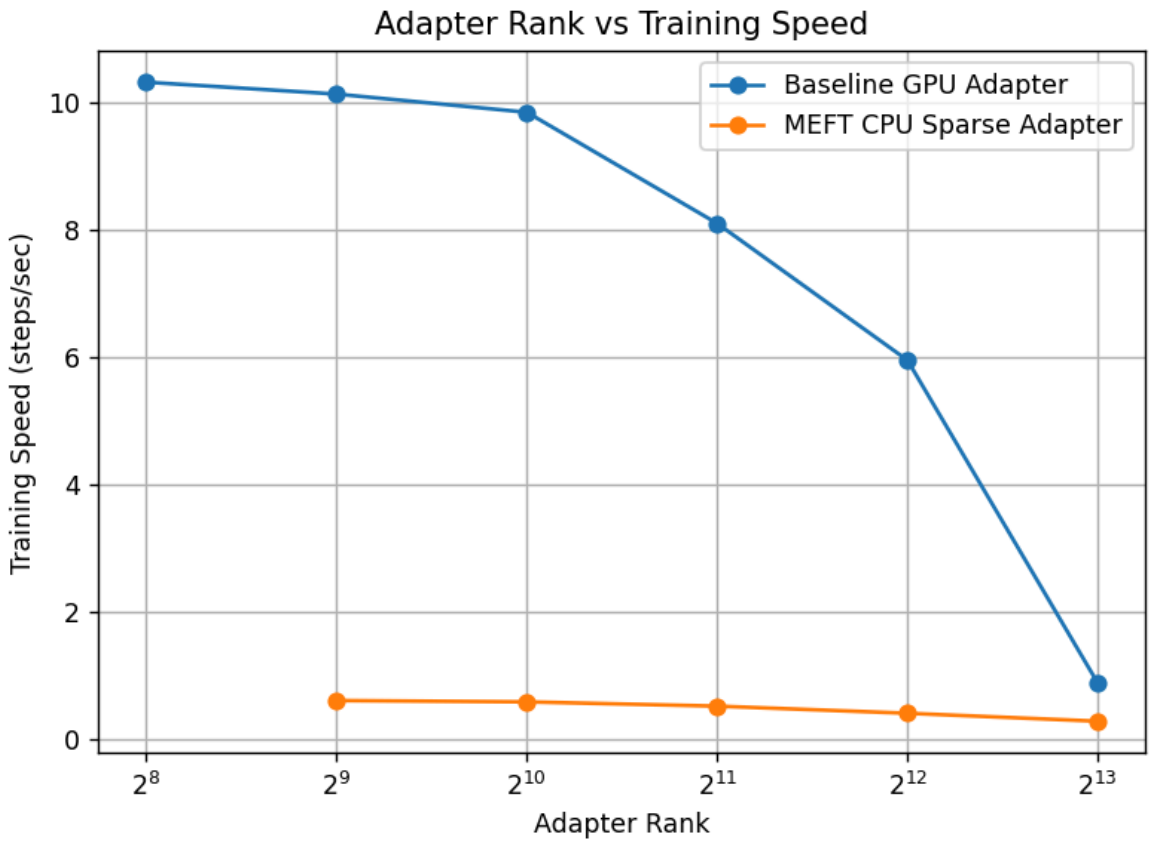
- Only selected weights are transferred to GPU

Training is performed using a simple loop with forward and backward passes, measuring memory and speed.

---

5. Results





VRAM Usage

- Baseline memory increases significantly with adapter rank
- MEFT memory remains nearly constant (~3.28 GB)

Example:

- Baseline (rank 8192): ~9102 MB
- MEFT (rank 8192): ~3280 MB

~64% reduction in GPU memory

Training Speed

- Baseline is significantly faster (GPU-only computation)
- MEFT is slower due to:
    - CPU → GPU data transfer
    - Top-K computation overhead

However:

- Baseline slows down as rank increases
- MEFT remains relatively stable

Key Observation

MEFT allows scaling adapter size without increasing GPU memory, at the cost of slower training speed.

---

6. Discussion and Future Work

This project demonstrates that MEFT effectively decouples adapter size from GPU memory usage through sparse computation.

Trade-offs

- Pros:
    - Significant reduction in GPU memory
    - Enables large adapter sizes
- Cons:
    - Slower training due to CPU-GPU communication
    - Additional computation for Top-K selection

Future Work

- Implement MoE routing for faster selection
- Optimize CPU-GPU transfer
- Evaluate on larger datasets and tasks
- Explore dynamic K selection

---

7. References

Hao et al.
MEFT: Memory-Efficient Fine-Tuning through Sparse Adapter
ACL 2024